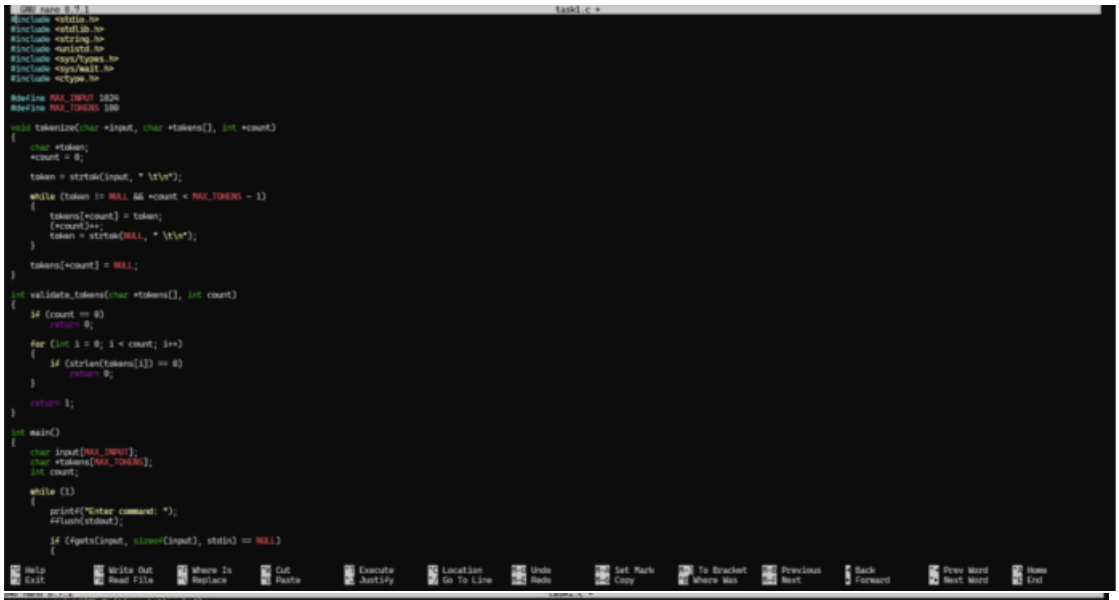
Skill – Week 4:

Task 1:



```
GNU nano 8.7.1                                  task1.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_TOKENS 100

void tokenize(char *input, char *tokens[], int *count)
{
    char *token;
    *count = 0;

    token = strtok(input, " \t\n");

    while (token != NULL && *count < MAX_TOKENS - 1)
    {
        tokens[*count] = token;
        (*count)++;
        token = strtok(NULL, " \t\n");
    }

    tokens[*count] = NULL;
}

int validate_tokens(char *tokens[], int count)
{
    if (count == 0)
        return 0;

    for (int i = 0; i < count; i++)
    {
        if (strlen(tokens[i]) == 0)
            return 0;
    }

    return 1;
}

int main()
{
    char input[MAX_INPUT];
    char *tokens[MAX_TOKENS];
    int count;

    while (1)
    {
        printf("Enter command: ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
        {
^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo    M-A Set Mark   M-] To Bracket  M-> Previous  ^B Back      ^Left Prev Word ^A Home
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo    M-6 Copy       ^Q Where Was    M-< Next      ^F Forward   ^Right Next Word ^E End
```



```
        printf("\nExiting shell...\n");
        break;
    }

    if (strlen(input) == 1 && input[0] == '\n')
    {
        printf("Empty command. Please enter a command.\n");
        continue;
    }

    tokenize(input, tokens, &count);

    if (!validate_tokens(tokens, count))
    {
        printf("Invalid command.\n");
        continue;
    }

    if (strcmp(tokens[0], "exit") == 0)
    {
        printf("Exiting shell...\n");
        break;
    }

    printf("Tokens:\n");

    for (int i = 0; i < count; i++)
    {
        printf("Token %d: %s\n", i + 1, tokens[i]);
    }

    pid_t pid = fork();

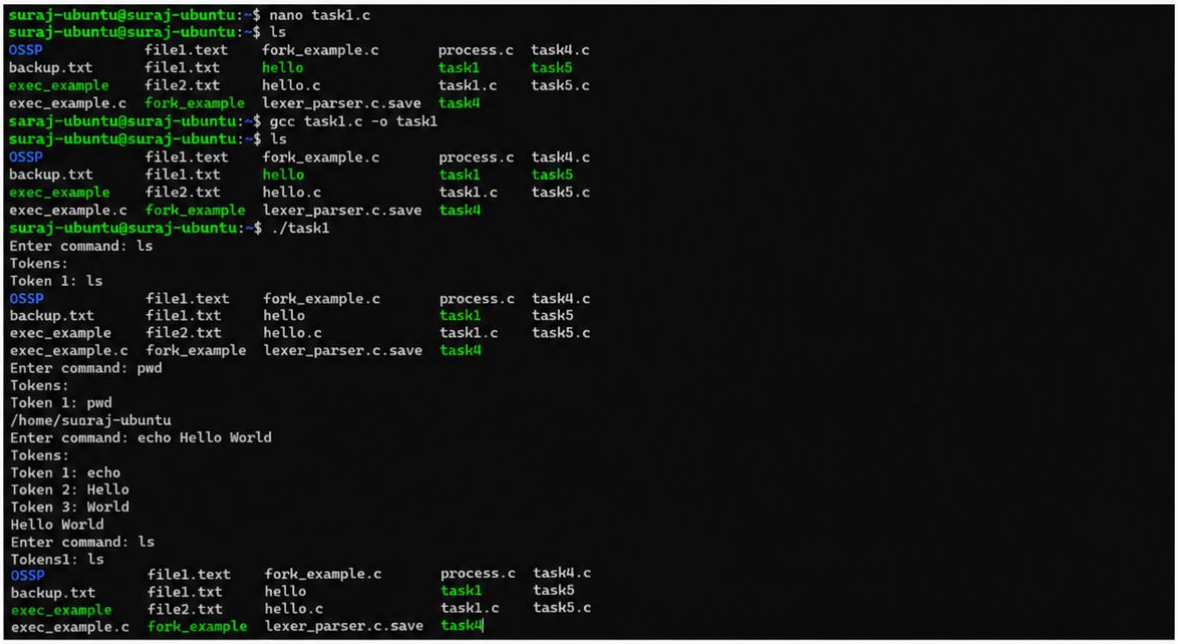
    if (pid < 0)
    {
        perror("Fork failed");
        continue;
    }

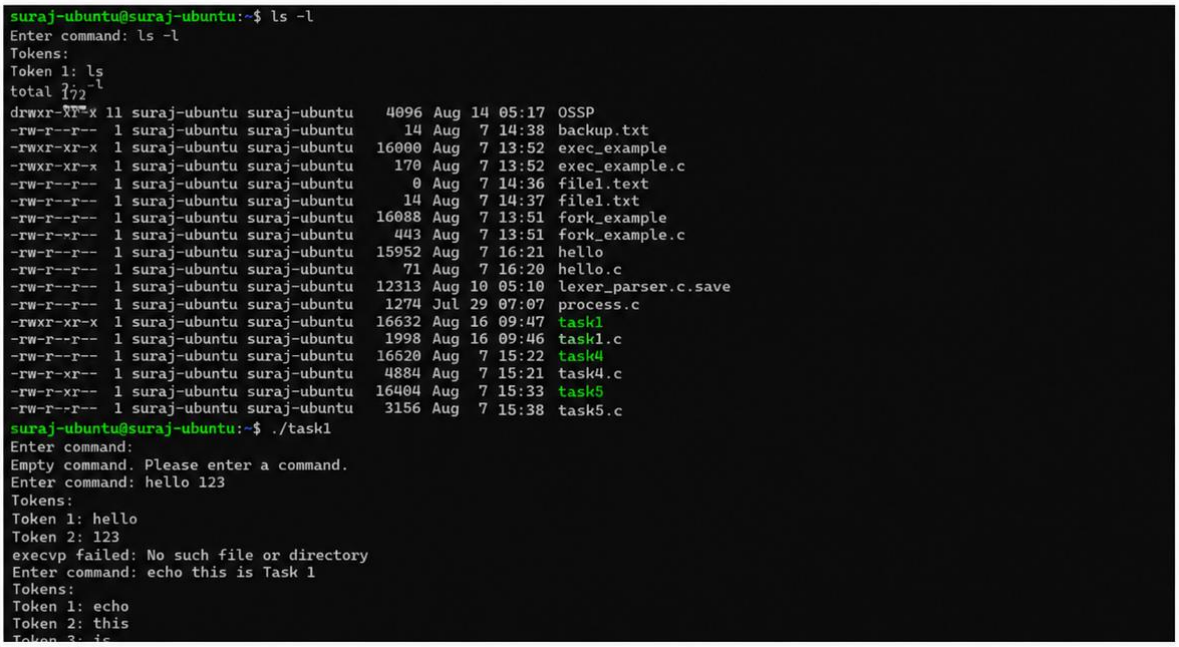
    if (pid == 0)
    {
        execvp(tokens[0], tokens);

        perror("execvp failed");
        exit(EXIT_FAILURE);
    }
    else
    {
        waitpid(pid, NULL, 0);
    }
}

return 0;

^G Help      ^O Write Out ^W Where Is  ^K Cut       ^T Execute   ^C Location  M-U Undo    M-A Set Mark   M-] To Bracket  M-> Previous  ^B Back      ^Left Prev Word ^A Home
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^/ Go To Line M-E Redo    M-6 Copy       ^Q Where Was    M-< Next      ^F Forward   ^Right Next Word ^E End
```







successfully demonstrates the core mechanics of a custom shell, focusing on parsing raw user input into structured tokens and validating them. By leveraging foundational system calls like fork(),

execvp(), and waitpid(), it establishes a robust workflow for isolated command execution and orderly process management.

TASK-2



```
GNU nano 8.7.1                                  task2.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_ARGS 100
#define MAX_LEN 1024

int parse_command(char *input, char *args[]) {
    int argc = 0;
    int in_quote = 0;
    char quote = '\0';
    char *p = input;

    while (*p) {
        while (*p == ' ' || *p == '\t' || *p == '\n')
            p++;

        if (*p == '\0')
            break;

        args[argc++] = p;

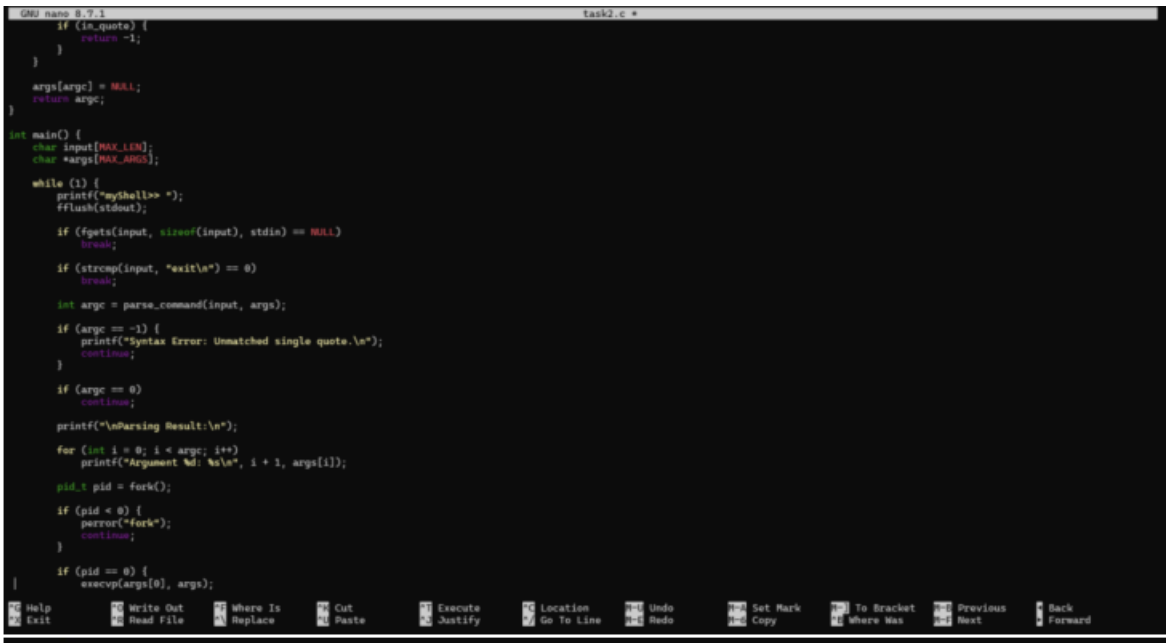
        while (*p) {
            if (!in_quote && (*p == '\'' || *p == '"')) {
                in_quote = 1;
                quote = *p;
                memmove(p, p + 1, strlen(p));
                continue;
            }

            if (in_quote && *p == quote) {
                in_quote = 0;
                memmove(p, p + 1, strlen(p));
                continue;
            }

            if (!in_quote && (*p == ' ' || *p == '\t' || *p == '\n')) {
                *p = '\0';
                p++;
                break;
            }

            p++;
        }

^G Help      ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark   M-] To Bracket  M-Q Previous   ^B Back
^X Exit      ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy       ^Q Where Was    M-W Next       ^F Forward
```



```
GNU nano 8.7.1                                  task2.c *
        if (in_quote) {
            return -1;
        }
    }

    args[argc] = NULL;
    return argc;
}

int main() {
    char input[MAX_LEN];
    char *args[MAX_ARGS];

    while (1) {
        printf("myShell>> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        if (strcmp(input, "exit\n") == 0)
            break;

        int argc = parse_command(input, args);

        if (argc == -1) {
            printf("Syntax Error: Unmatched single quote.\n");
            continue;
        }

        if (argc == 0)
            continue;

        printf("\nParsing Result:\n");

        for (int i = 0; i < argc; i++)
            printf("Argument %d: %s\n", i + 1, args[i]);

        pid_t pid = fork();

        if (pid < 0) {
            perror("fork");
            continue;
        }

        if (pid == 0) {
            execvp(args[0], args);

^G Help      ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark   M-] To Bracket  M-Q Previous   ^B Back
^X Exit      ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy       ^Q Where Was    M-W Next       ^F Forward
```



```
        if (pid == 0) {
            execvp(args[0], args);
            perror("execvp");
            exit(1);
        } else {
            printf("\nChild PID: %d\n", pid);

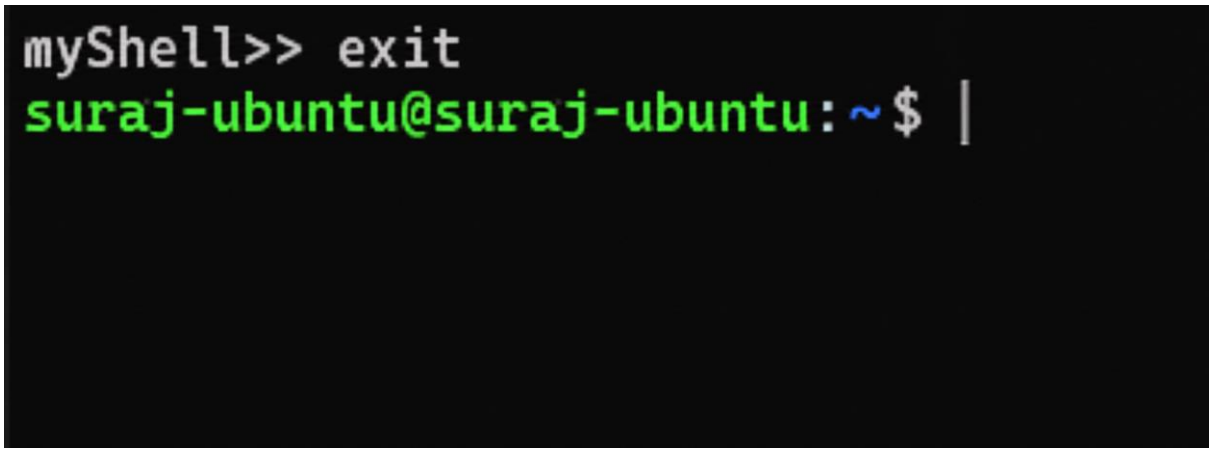
            int status;
            waitpid(pid, &status, 0);

            printf("\nCommand execution completed.\n\n");
        }
    }

    return 0;
}

^G Help      ^O Write Out   ^W Where Is    ^K Cut         ^T Execute     ^C Location    M-U Undo       M-A Set Mark   M-] To Bracket  M-Q Previous   ^B Back
^X Exit      ^R Read File   ^\ Replace     ^U Paste       ^J Justify     ^/ Go To Line  M-E Redo       M-6 Copy       ^Q Where Was    M-W Next       ^F Forward
```

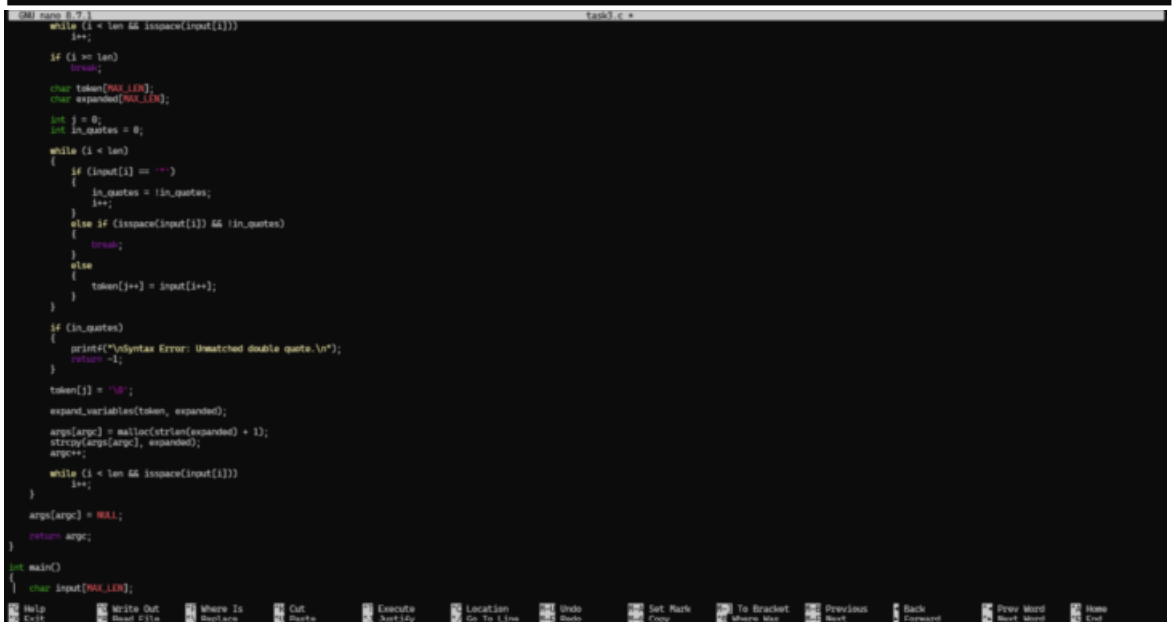


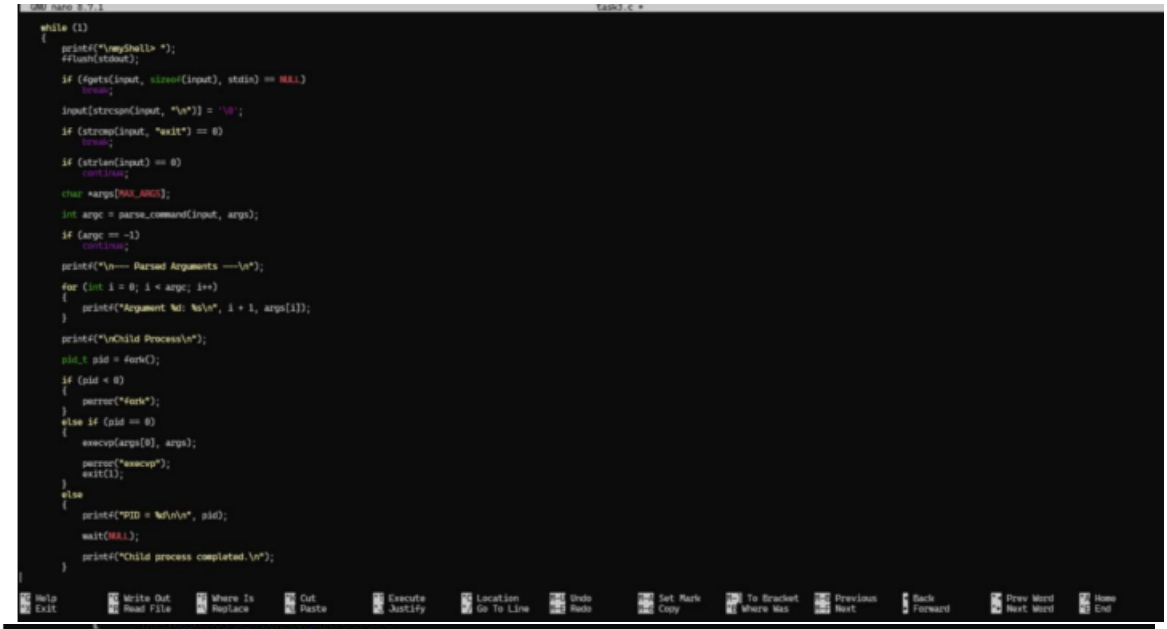


successfully implemented single-quote handling, argument parsing, literal content preservation, and syntax-error detection. It also demonstrated process creation using fork (), command execution using execvp (), and synchronization using waitpid ().

Task 3:


GNU nano 8.7.1
task3.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <ctype.h>
#define MAX_ARGS 100
#define MAX_LEN 1024
void expand_variables(char *src, char *dest)
{
int i = 0, j = 0;
while (src[i] != '\0')
{
if (src[i] == '$')
{
i++;
char var[100];
int k = 0;
while (isalnum(src[i]) || src[i] == '_')
{
var[k++] = src[i++];
}
var[k] = '\0';
char *value = getenv(var);
if (value != NULL)
{
strcpy(&dest[j], value);
j += strlen(value);
}
}
else
{
dest[j++] = src[i++];
}
}
dest[j] = '\0';
}
int parse_command(char *input, char *args[])
{
int argc = 0;
int i = 0;
int len = strlen(input);
while (i < len)
{
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-P Previous ^B Back ^Left Prev Word ^A Home
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was M-N Next ^F Forward ^Right Next Word ^E End


GNU nano 8.7.1
task1.c *
while (i < len && isspace(input[i]))
i++;
if (i >= len)
break;
char token[MAX_LEN];
char expanded[MAX_LEN];
int j = 0;
int in_quotes = 0;
while (i < len)
{
if (input[i] == '"')
{
in_quotes = !in_quotes;
i++;
}
else if (isspace(input[i]) && !in_quotes)
{
break;
}
else
{
token[j++] = input[i++];
}
}
if (in_quotes)
{
printf("\nSyntax Error: Unmatched double quote.\n");
return -1;
}
token[j] = '\0';
expand_variables(token, expanded);
args[argc] = malloc(strlen(expanded) + 1);
strcpy(args[argc], expanded);
argc++;
while (i < len && isspace(input[i]))
i++;
}
args[argc] = NULL;
return argc;
}
int main()
{
char input[MAX_LEN];
^G Help ^O Write Out ^W Where Is ^K Cut ^T Execute ^C Location M-U Undo M-A Set Mark M-] To Bracket M-P Previous ^B Back ^Left Prev Word ^A Home
^X Exit ^R Read File ^\ Replace ^U Paste ^J Justify ^/ Go To Line M-E Redo M-6 Copy ^Q Where Was M-N Next ^F Forward ^Right Next Word ^E End



```
    while (1)
    {
        printf("\nmyShell> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        char *args[MAX_ARGS];

        int argc = parse_command(input, args);

        if (argc == -1)
            continue;

        printf("\n--- Parsed Arguments ---\n");

        for (int i = 0; i < argc; i++)
        {
            printf("Argument %d: %s\n", i + 1, args[i]);
        }

        printf("\nChild Process\n");

        pid_t pid = fork();

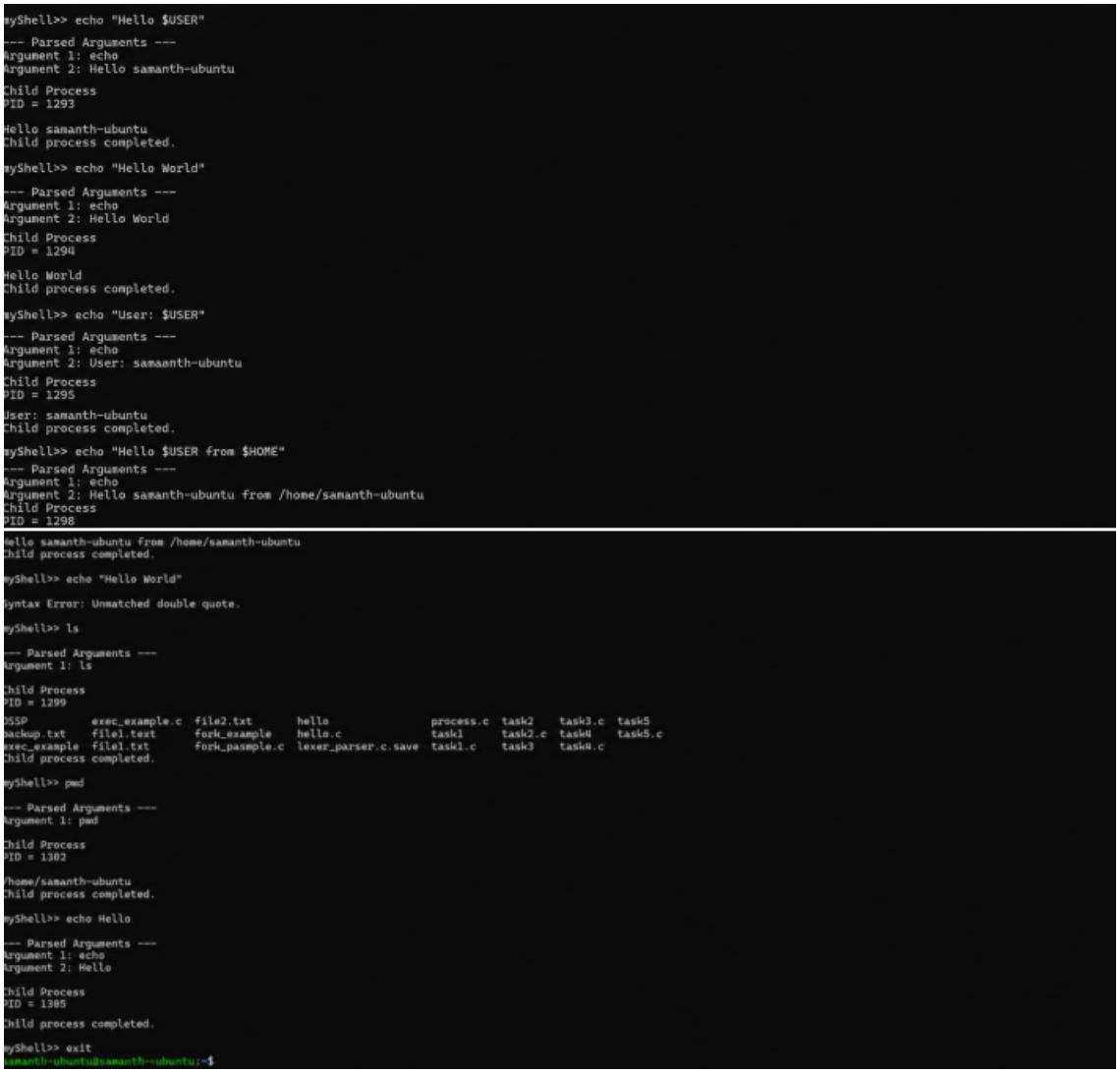
        if (pid < 0)
        {
            perror("fork");
        }
        else if (pid == 0)
        {
            execvp(args[0], args);

            perror("execvp");
            exit(1);
        }
        else
        {
            printf("PID = %d\n\n", pid);

            wait(NULL);

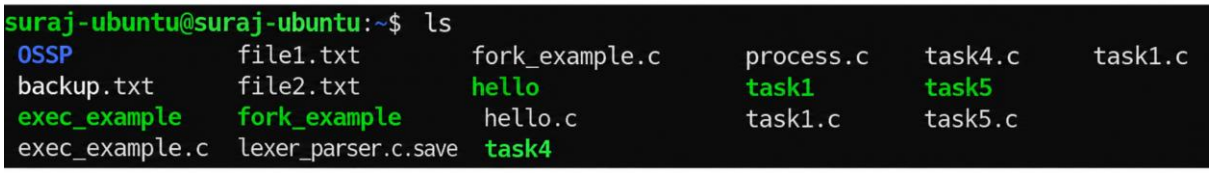
            printf("Child process completed.\n");
        }
```

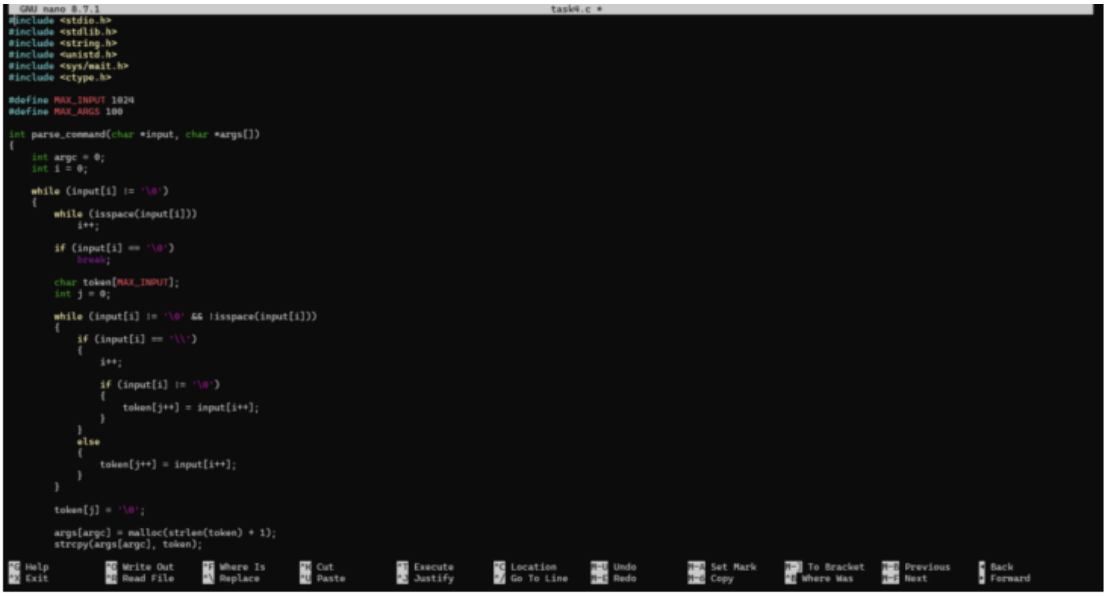
```
^G Help          ^O Write Out     ^F Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo
```



successfully implemented double-quote handling, space preservation, variable expansion, and nested token parsing in the shell. It also validated unmatched quotes and executed valid commands using child processes with fork () and execvp ().

Task 4:





```
 GNU nano 8.7.1                                                task4.c *
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <ctype.h>

#define MAX_INPUT 1024
#define MAX_ARGS 100

int parse_command(char *input, char *args[])
{
    int argc = 0;
    int i = 0;

    while (input[i] != '\0')
    {
        while (isspace(input[i]))
            i++;

        if (input[i] == '\0')
            break;

        char token[MAX_INPUT];
        int j = 0;

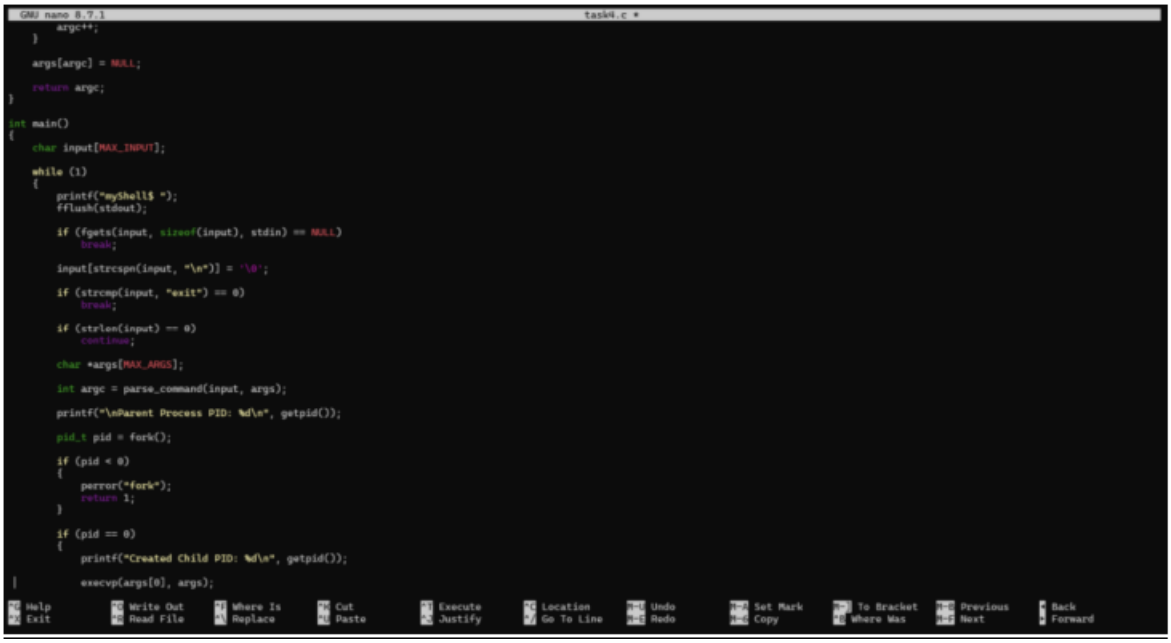
        while (input[i] != '\0' && !isspace(input[i]))
        {
            if (input[i] == '\\')
            {
                i++;

                if (input[i] != '\0')
                {
                    token[j++] = input[i++];
                }
            }
            else
            {
                token[j++] = input[i++];
            }
        }

        token[j] = '\0';

        args[argc] = malloc(strlen(token) + 1);
        strcpy(args[argc], token);

^G Help          ^O Write Out     ^W Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo         M-A Set Mark     M-] To Bracket   M-Q Previous     ^B Back
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo         M-6 Copy         ^Q Where Was     M-W Next         ^F Forward
```



```
 GNU nano 8.7.1                                                task4.c *
        argc++;
    }

    args[argc] = NULL;

    return argc;
}

int main()
{
    char input[MAX_INPUT];

    while (1)
    {
        printf("myShell$ ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';

        if (strcmp(input, "exit") == 0)
            break;

        if (strlen(input) == 0)
            continue;

        char *args[MAX_ARGS];

        int argc = parse_command(input, args);

        printf("\nParent Process PID: %d\n", getpid());

        pid_t pid = fork();

        if (pid < 0)
        {
            perror("fork");
            return 1;
        }

        if (pid == 0)
        {
            printf("Created Child PID: %d\n", getpid());

            execvp(args[0], args);

^G Help          ^O Write Out     ^W Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo         M-A Set Mark     M-] To Bracket   M-Q Previous     ^B Back
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo         M-6 Copy         ^Q Where Was     M-W Next         ^F Forward
```



```
            perror(args[0]);
            exit(2);
        }
        else
        {
            printf("Child Process PID: %d\n", pid);

            int status;

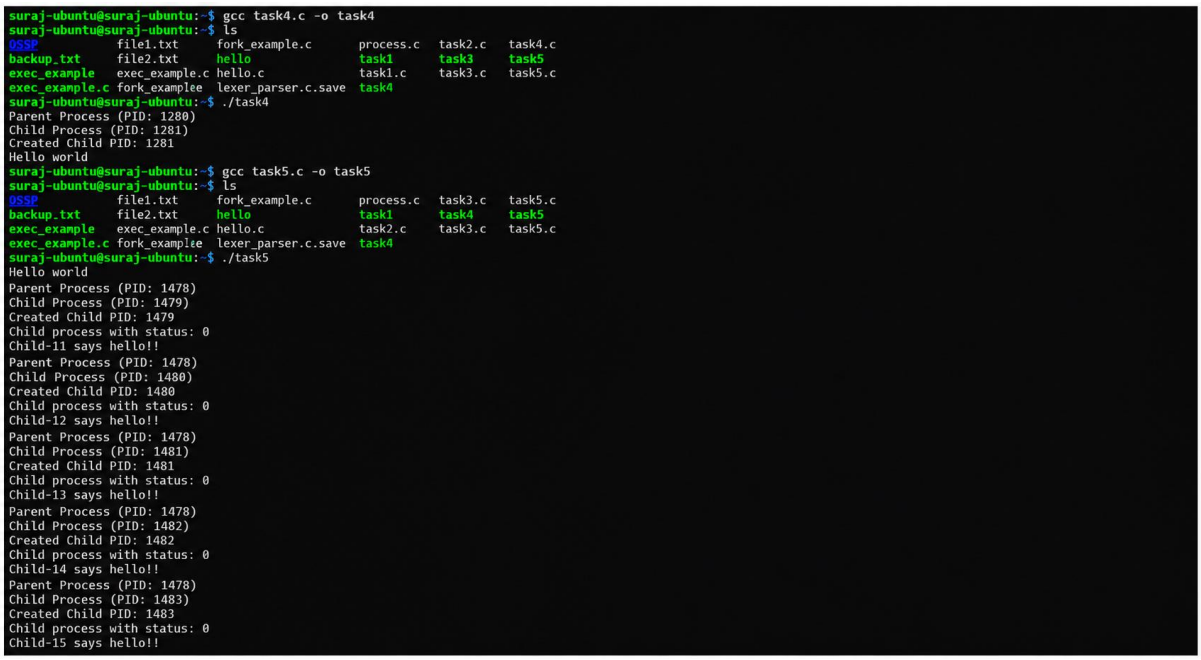
            waitpid(pid, &status, 0);

            if (WIFEXITED(status))
            {
                printf("Child exited with status: %d\n",
                       WEXITSTATUS(status));
            }
        }

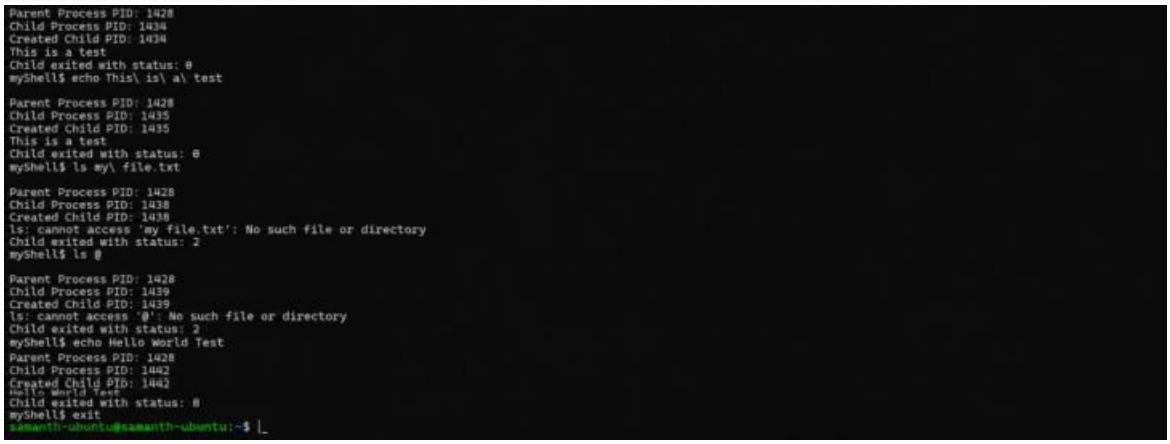
        for (int i = 0; i < argc; i++)
            free(args[i]);
    }

    return 0;
}
```

```
^G Help          ^O Write Out     ^F Where Is      ^K Cut           ^T Execute       ^C Location      M-U Undo
^X Exit          ^R Read File     ^\ Replace       ^U Paste         ^J Justify       ^/ Go To Line    M-E Redo
```



```
suraj-ubuntu@suraj-ubuntu:~$ gcc task4.c -o task4
suraj-ubuntu@suraj-ubuntu:~$ ls
OSSP            file1.txt      fork_example.c       process.c   task2.c   task4.c
backup.txt      file2.txt      hello                task1       task3     task5
exec_example    exec_example.c hello.c              task1.c     task3.c   task5.c
exec_example.c fork_examplee  lexer_parser.c.save  task4
suraj-ubuntu@suraj-ubuntu:~$ ./task4
Parent Process (PID: 1280)
Child Process (PID: 1281)
Created Child PID: 1281
Hello world
suraj-ubuntu@suraj-ubuntu:~$ gcc task5.c -o task5
suraj-ubuntu@suraj-ubuntu:~$ ls
OSSP            file1.txt      fork_example.c       process.c   task3.c   task5.c
backup.txt      file2.txt      hello                task1       task4     task5
exec_example    exec_example.c hello.c              task2.c     task3.c   task5.c
exec_example.c fork_examplee  lexer_parser.c.save  task4
suraj-ubuntu@suraj-ubuntu:~$ ./task5
Hello world
Parent Process (PID: 1478)
Child Process (PID: 1479)
Created Child PID: 1479
Child process with status: 0
Child-11 says hello!!
Parent Process (PID: 1478)
Child Process (PID: 1480)
Created Child PID: 1480
Child process with status: 0
Child-12 says hello!!
Parent Process (PID: 1478)
Child Process (PID: 1481)
Created Child PID: 1481
Child process with status: 0
Child-13 says hello!!
Parent Process (PID: 1478)
Child Process (PID: 1482)
Created Child PID: 1482
Child process with status: 0
Child-14 says hello!!
Parent Process (PID: 1478)
Child Process (PID: 1483)
Created Child PID: 1483
Child process with status: 0
Child-15 says hello!!
```



```
Parent Process PID: 1428
Child Process PID: 1434
Created Child PID: 1434
This is a test
Child exited with status: 0
myShell$ echo This\ is\ a\ test

Parent Process PID: 1428
Child Process PID: 1435
Created Child PID: 1435
This is a test
Child exited with status: 0
myShell$ ls my\ file.txt

Parent Process PID: 1428
Child Process PID: 1438
Created Child PID: 1438
ls: cannot access 'my file.txt': No such file or directory
Child exited with status: 2
myShell$ ls @

Parent Process PID: 1428
Child Process PID: 1439
Created Child PID: 1439
ls: cannot access '@': No such file or directory
Child exited with status: 2
myShell$ echo Hello World Test
Parent Process PID: 1428
Child Process PID: 1442
Created Child PID: 1442
Hello World Test
Child exited with status: 0
myShell$ exit
samanth-ubuntu@samanth-ubuntu:~$ _
```

successfully implemented escape sequences, escaped spaces, and special character handling while preserving the correct command arguments. It also created child processes using fork(), executed commands using execvp(), and handled child process exit statuses and errors.